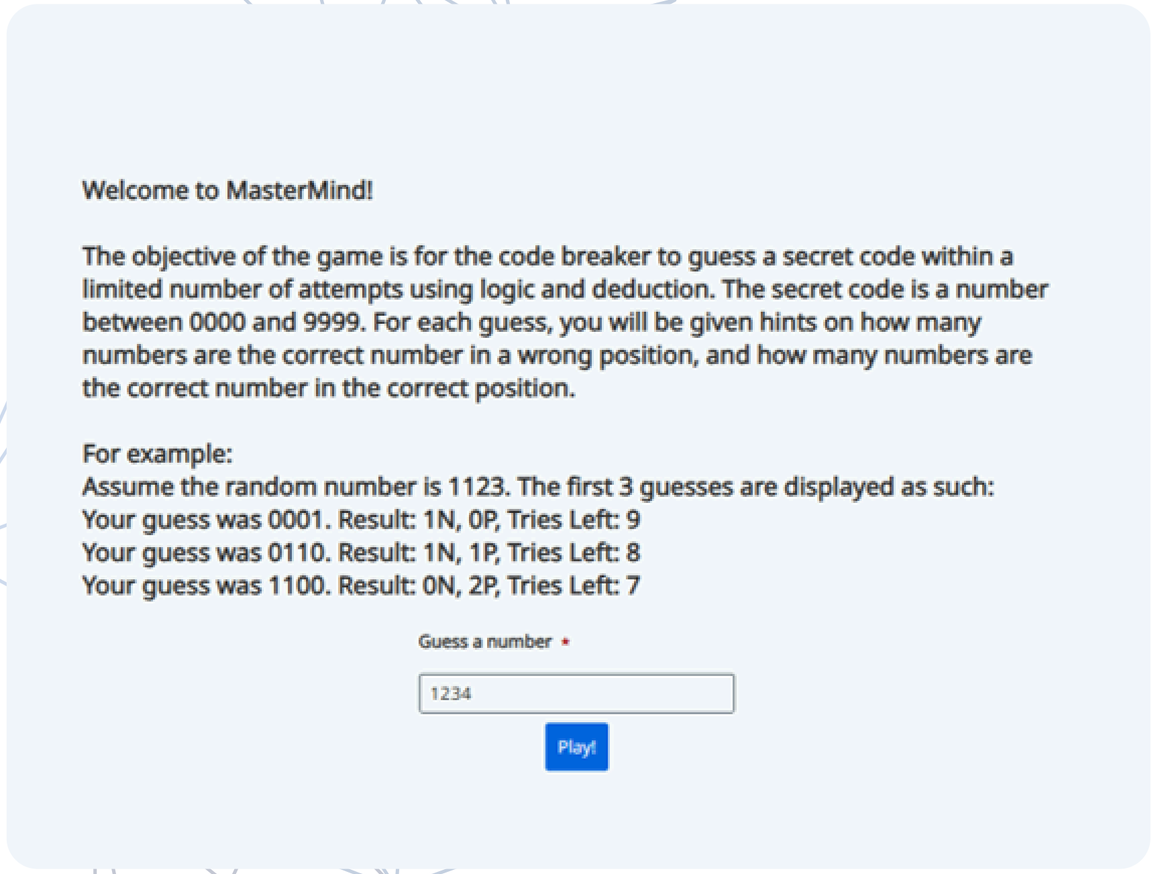


Mastermind Automation App

- inspired by the classic code-cracking game Mastermind
- created using UiPath Studio Web
- uses variables, conditions, loops, user input and feedback logic

Objectives

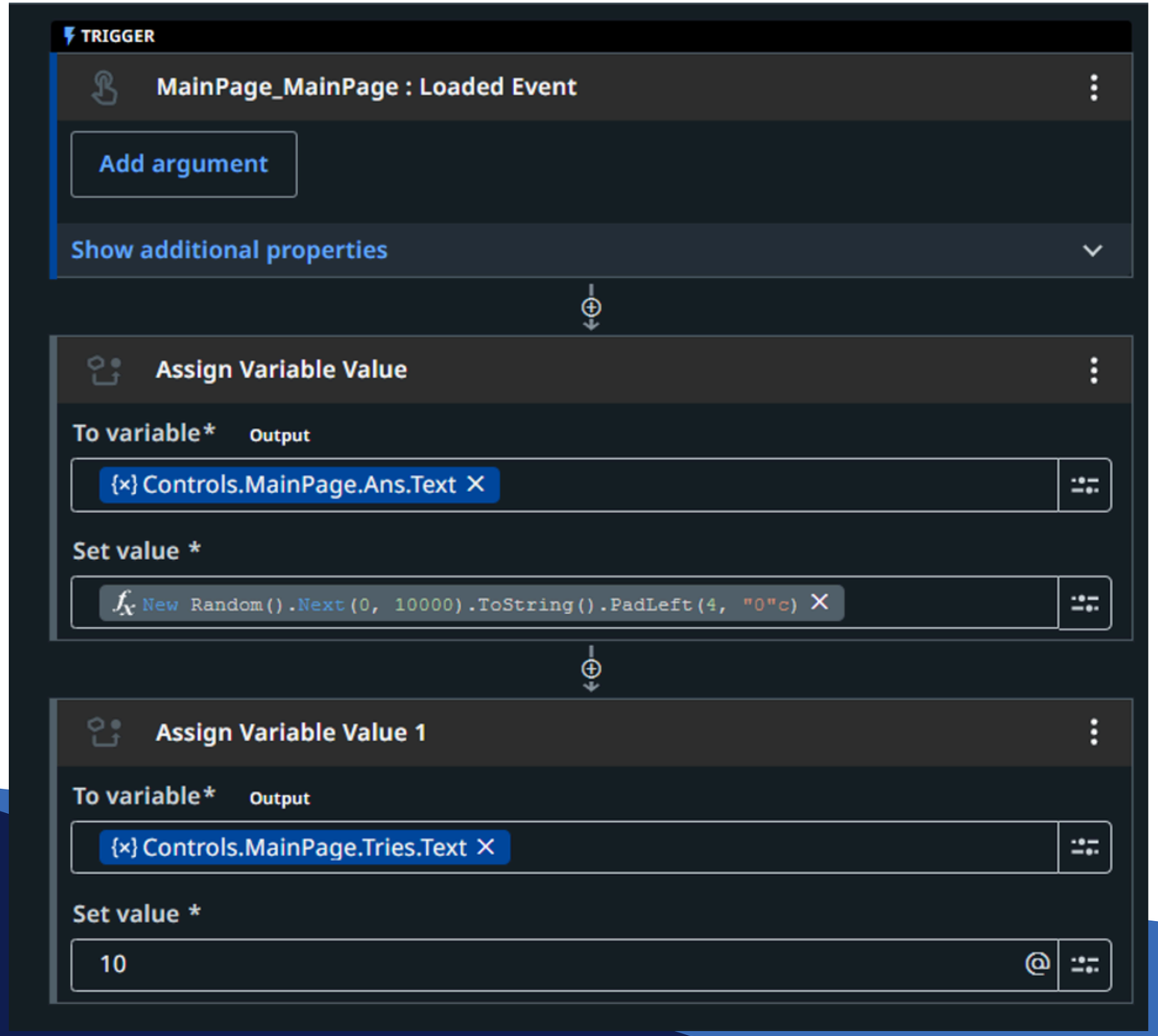
- The goal was to create an interactive Mastermind game where the user input their guesses, and the app will give feedback on their guesses after each try.

- There are two outcomes: either the user successfully guesses the number code or they run out of tries and lose the game.

Python Planning

Planning was done on local code space using Python.

```python
userguess = 'user input'
ans = 'ans text box'
N = 0 #correct number wrong position
P = 0 #correct number correct position
ansleft = ""
guessleft = ""
triesleft = 10
idx = 0
fourloop = 0

if userguess == ans:
    print(f"You've won! The number was {ans}.")

else:
    triesleft = triesleft - 1

    if triesleft == 0:
        print(f"Game Over! The number was {ans}.")

    else:

        while fourloop < 4:
            if userguess[fourloop] == ans[fourloop]: #if correct number in correct position then increase P count
                P = P + 1
            else: #not matching, keep these terms to check later
                guessleft = guessleft + userguess[fourloop]
                ansleft = ansleft + ans[fourloop]
            fourloop = fourloop + 1

        for num in guessleft:
            if num in ansleft:
                N = N + 1
                idx = ansleft.index(num)
                ansleft = ansleft[:idx] + ansleft[idx+1:]

        print(f"Your guess was {userguess}. Result: {N}N, {P}P, Tries Left: {triesleft}")

```



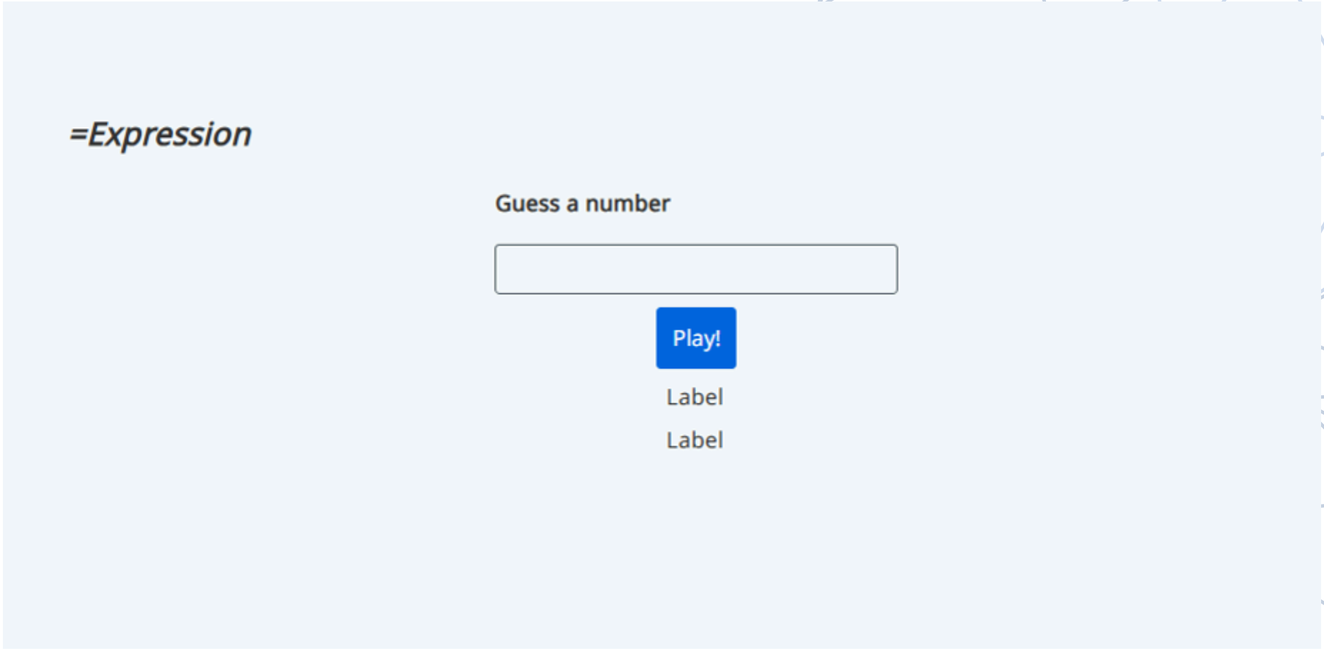
Initialisation

When the page loads, a 4-digit code is randomly generated, and a total of 10 tries will be given to the user.

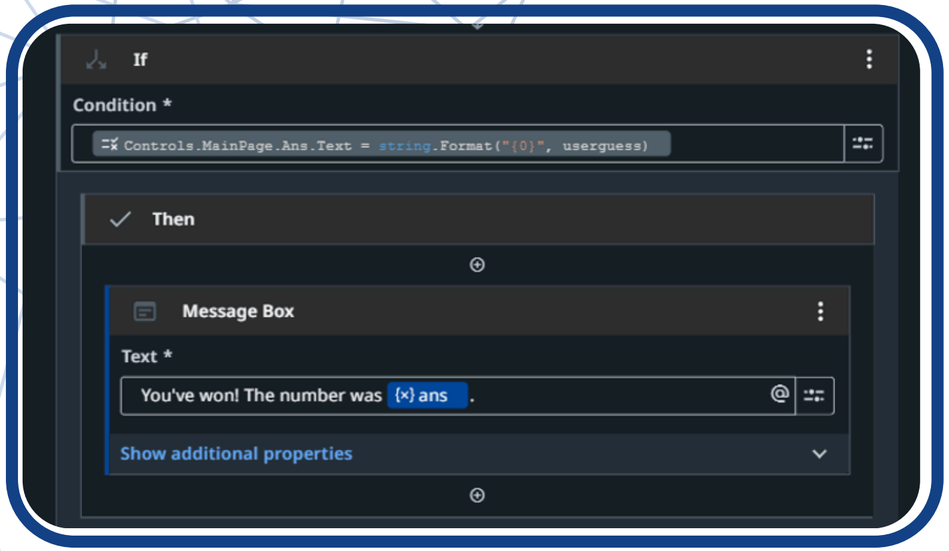
Input

A box is given for users to input their guesses, and the guess is submitted when the button “Play!” is pressed.

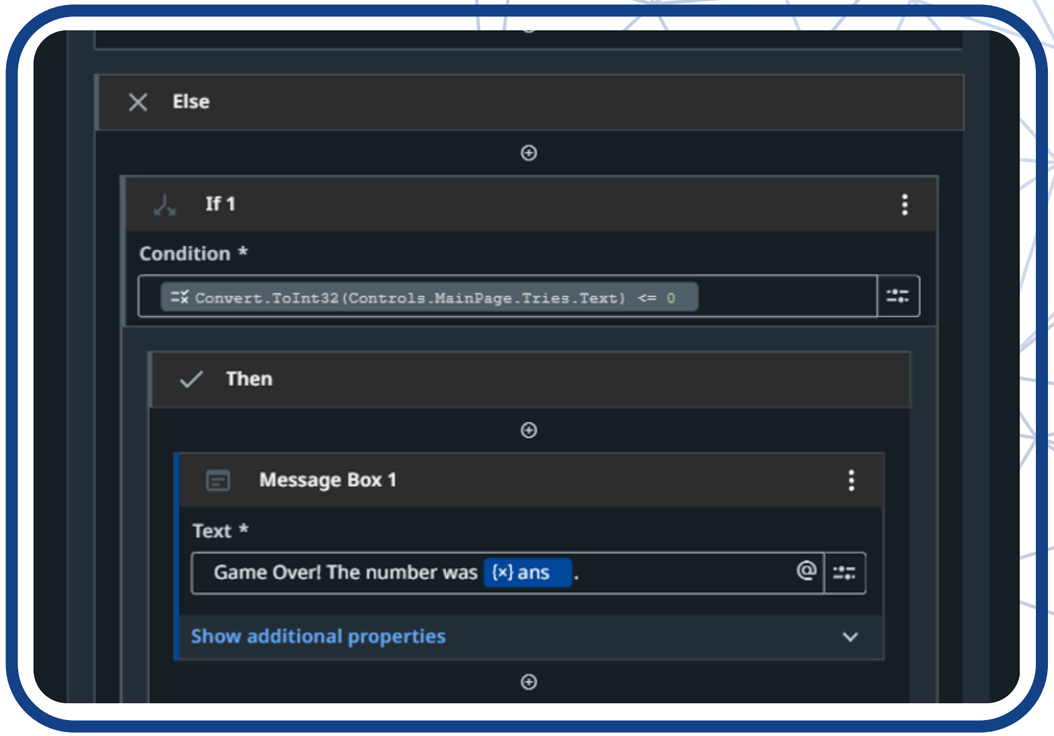
The expression above the box helps explain the game to users who are unfamiliar to it.



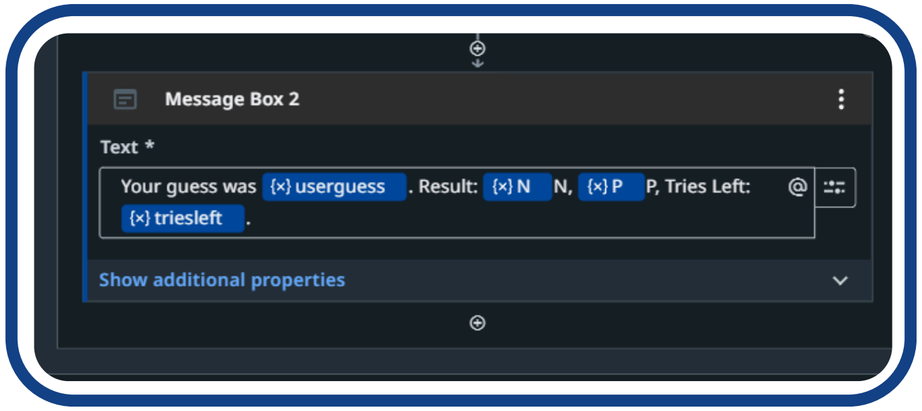
Output



Correct guess within 10 tries

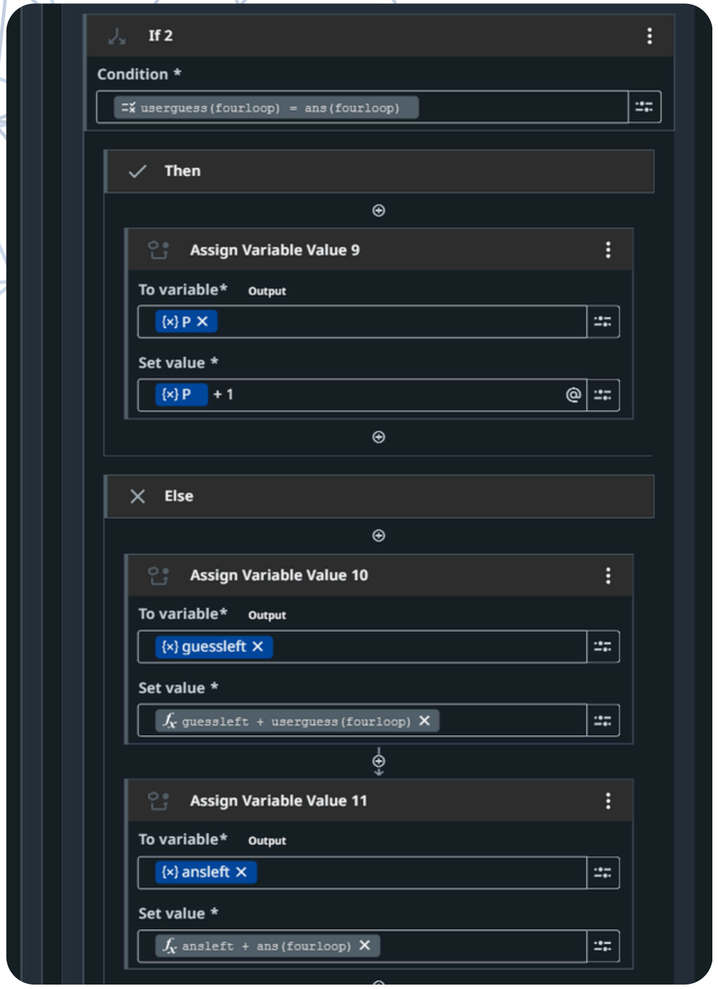


Running out of tries



Wrong guess with tries remaining

Logic



The user's guess will be compared to the correct answer, then P is incremented for each correct number in the right position.

For the remaining numbers in the user's guess, they will be individually checked to see if they exist in the answer, and if so, N will be incremented.

Challenges

Platform It took some time learning the UiPath interface while reading the documentations.

Variables Unlike usual python coding where variables are initialised, I had to also store the value of some variables, and extract it at later parts.

Workflow Structuring the workflow took great effort to ensure the conditions and loops are working properly for the game to be played according to the given rule set.

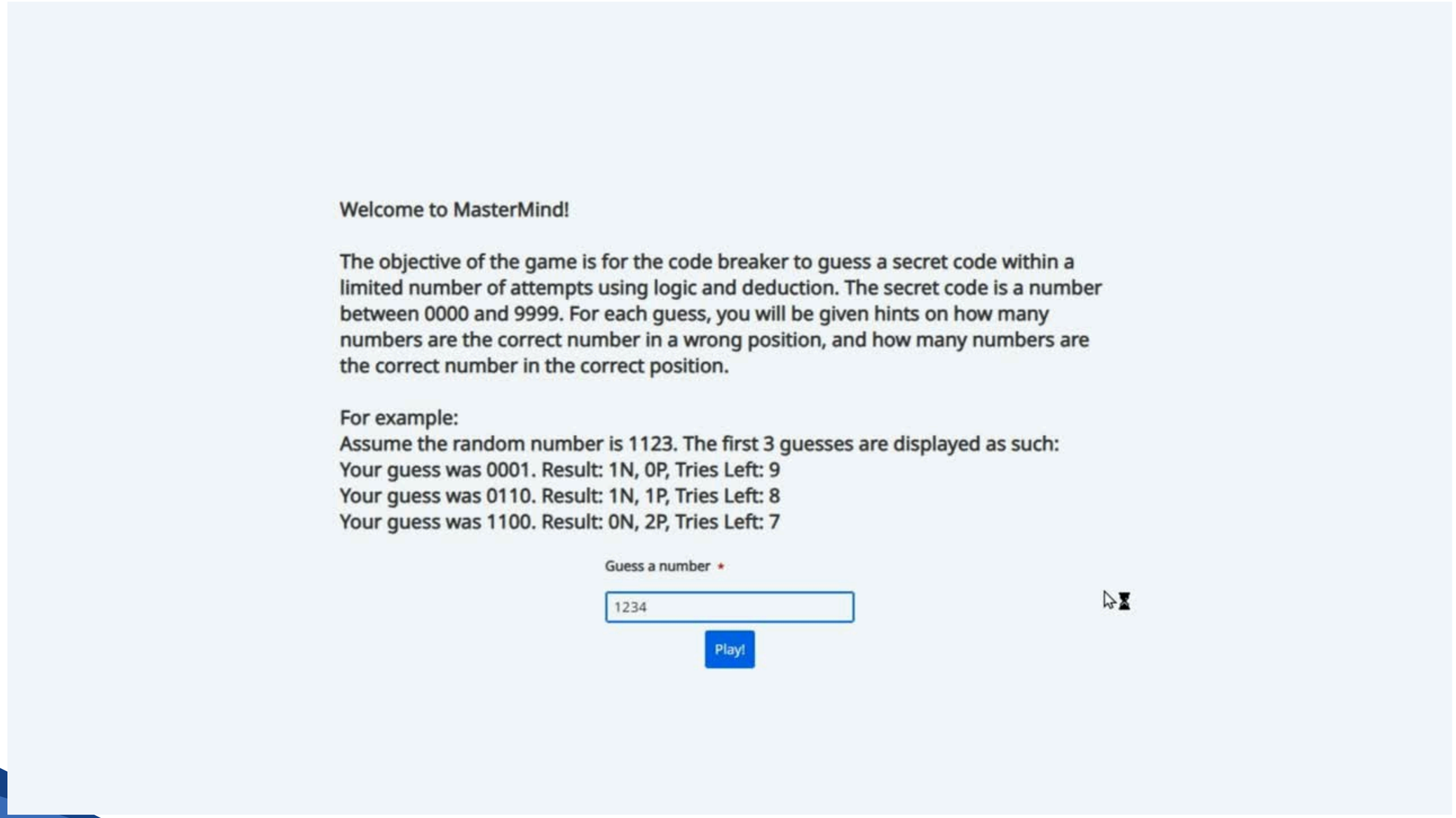
Possible improvements

Score A log history could be added to track previous guesses, along with a digital space to use as a space to take notes.

Difficulty Add different difficulty levels, such as varying number of digits for the code.

Hint Add a hint button for in case the user is very stuck, or integrate an AI system to help out when the user desires.

App Demo


Welcome to MasterMind!
The objective of the game is for the code breaker to guess a secret code within a limited number of attempts using logic and deduction. The secret code is a number between 0000 and 9999. For each guess, you will be given hints on how many numbers are the correct number in a wrong position, and how many numbers are the correct number in the correct position.
For example:
Assume the random number is 1123. The first 3 guesses are displayed as such:
Your guess was 0001. Result: 1N, 0P, Tries Left: 9
Your guess was 0110. Result: 1N, 1P, Tries Left: 8
Your guess was 1100. Result: 0N, 2P, Tries Left: 7
Guess a number *
1234
Play!